

DEVOPS · SYSTEM ADMINISTRATION

Inception

A learner's guide to Docker, containers & multi-service infrastructure

A teaching companion for the 42 "Inception" project.

Built to explain the *why* behind every requirement — from the Docker engine architecture down to PID 1, TLS, named volumes and the bonus stack.

This guide teaches concepts and points the way. It does **not** hand you the finished answer.

Docker

docker-compose

NGINX + TLS

WordPress + php-fpm

MariaDB

Redis

FTP

Adminer

Contents

0 · How to use this guide	4
1 · What is this project, really?	5
The problem · what you build · what you learn · success criteria	
2 · The mental models you need first	9
VM vs container · the kernel · namespaces & cgroups · images vs containers	
3 · Docker architecture, deep	14
client / daemon / containerd / runc · the layered filesystem · the build cache	
4 · The Dockerfile, line by line	20
every instruction · PID 1 · why no <code>tail -f</code> · daemons & foreground	
5 · Networking & volumes	27
bridge networks · DNS between containers · named volumes vs bind mounts	
6 · docker-compose	32
services · the compose file anatomy · secrets · env files	
7 · The three mandatory services	37
MariaDB · WordPress + php-fpm · NGINX + TLS · how they talk	
8 · The Makefile & the host VM	44
9 · The bonus stack	46
Redis · FTP · static site · Adminer · your own service	
10 · The command reference	50
11 · A suggested roadmap & the defense	53

CHAPTER 0

How to use this guide

This is a teacher, not an answer key. By the end you should be able to walk up to a whiteboard and explain every requirement in the subject — and then write the files yourself.

You have a strong low-level background already. You wrote `minishell`, so you understand processes, `fork`, `exec`, pipes, file descriptors and signals. You wrote `cub3d`, so you understand building something with many moving parts that must cooperate. **That foundation is exactly what makes this project click** — Docker is, at heart, a clever arrangement of Linux processes, namespaces and filesystems. You are not learning magic; you are learning which kernel features the magic is made of.

The contract this guide makes with you

- **It explains concepts from zero** and then goes deep, so nothing is a black box.
- **It shows you every command** you will touch, and explains what each flag does.
- **It gives diagrams** for the architecture, the build process, the network, and the request path.
- **It does not write your Dockerfiles, your compose file, or your scripts for you.** Where you would expect "the answer", you instead get the questions to ask yourself, the pitfalls to avoid, and the documentation to read. That is deliberate — at the defense you must explain your own work, and copied code that you cannot justify is an automatic fail.

WHY "GO DEEPER"?

The project is literally named after the film *Inception*: "we need to go deeper." The joke is that you build containers (a layer of virtualization) inside a virtual machine (another layer). The pedagogical point is the same: don't stop at "it works." Understand the layer beneath the one you're standing on.

How to read it

Read chapters 1–3 before touching a keyboard — they build the mental model. Then read chapter 4 (Dockerfile) and 5 (networking/volumes) *while* experimenting in a throwaway folder. Chapters 6–8 map directly onto the files you must submit. Chapter 9 is the bonus. Chapter 10 is a reference to keep open. Chapter 11 is the order to actually build things in.

A WORKING HABIT

Keep a scratch directory and a real terminal open. Every time this guide names a command, run it on something tiny (e.g. `docker run hello-world`, then inspect it). Reading about `docker ps` teaches you nothing; running it and reading every column teaches you everything.

CHAPTER 1

What is this project, really?

1.1 The one-sentence version

You will run a real, working WordPress website — served over HTTPS — where every piece (the web server, the PHP application, the database) lives in its own isolated container, and all of it is started by a single `make` command inside a virtual machine.

1.2 The problem we are solving

Imagine you want to host a website. The naive way is to take one computer, install NGINX, install PHP, install a database, edit a dozen config files by hand, and hope you remember what you did. Six months later you need a second server, or your laptop dies, or a teammate needs the same setup. You are now stuck: the configuration lives only in your head and in scattered files on a machine you can't perfectly reproduce.

This is the **"it works on my machine"** problem, and it is one of the central problems of system administration and DevOps. The infrastructure is not **reproducible**, not **isolated**, and not **portable**.

Containers solve this. The entire definition of each service — what base OS, what packages, what config, what command to run — is written down as code (a `Dockerfile`). The relationships between services — who talks to whom, what storage they share, what ports are exposed — is also written down as code (a `docker-compose.yml`). Now the whole stack is:

- **Reproducible** — anyone runs `make` and gets a byte-for-byte equivalent stack.
- **Isolated** — the database can't see the web server's files; a crash in one doesn't take down the others.
- **Portable** — it runs the same on your laptop, a teammate's machine, or a cloud server.
- **Versioned** — the infrastructure lives in Git alongside the code.

THIS IS "INFRASTRUCTURE AS CODE"

The big idea you are learning is that *infrastructure can be described in files, committed to version control, and rebuilt on demand*. Inception is your first hands-on encounter with it.

1.3 What you actually build

Three mandatory containers, two persistent volumes, one private network, one endpoint:

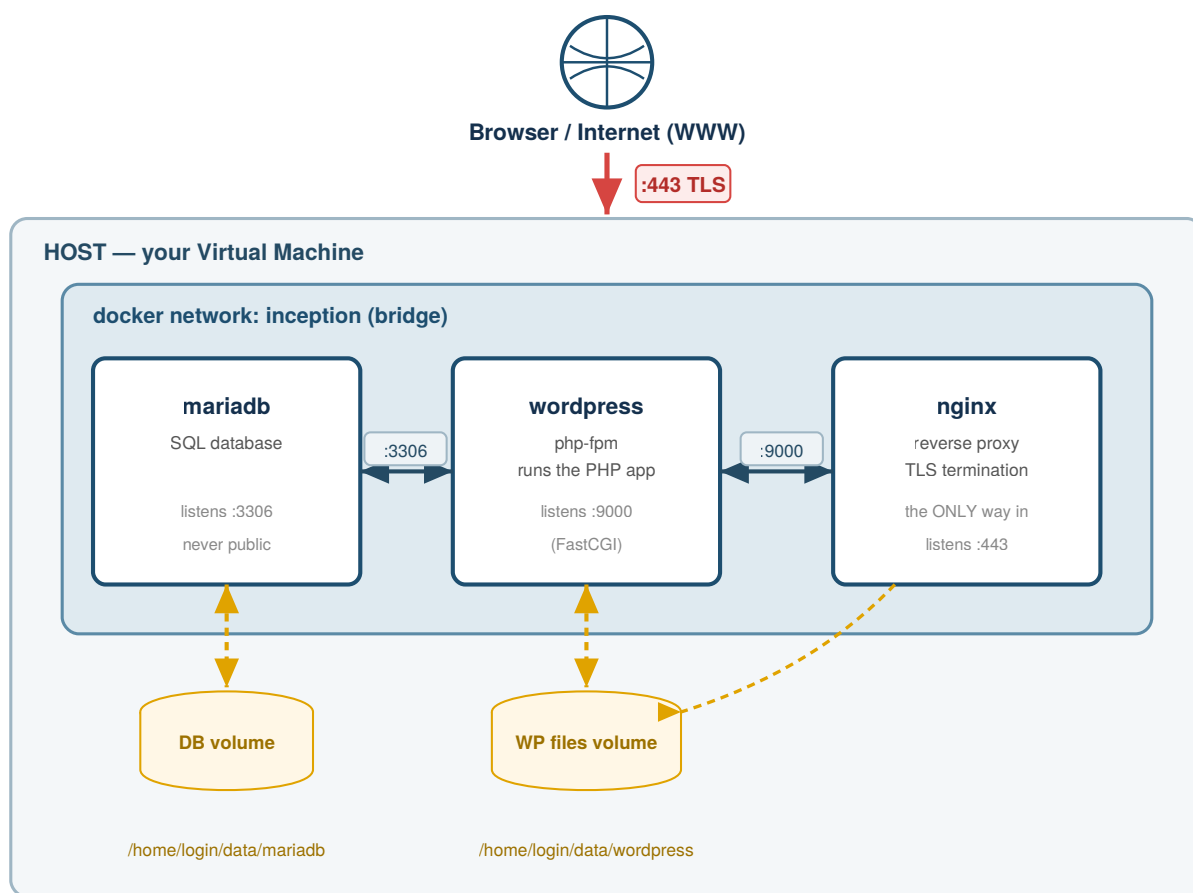


Figure 1.1 — The mandatory Inception infrastructure. Traffic enters **ONLY** through **NGINX** on port 443. **NGINX** forwards PHP requests to **WordPress** (php-fpm) on 9000; **WordPress** queries **MariaDB** on 3306. Two named volumes persist the data on the host.

1.4 Read Figure 1.1 like an engineer

Trace one request through the whole stack. This single mental walk-through explains *why* the architecture is shaped the way it is:

1. A browser asks for `https://login.42.fr`. The request arrives at the host on **port 443** and is handed to the **NGINX** container — the only door into the system.
2. **NGINX** terminates TLS (it holds the certificate and does the encryption/decryption). If the request is for a static file it serves it directly. If it's for a `.php` page, **NGINX** cannot run PHP itself — so it forwards the request over the FastCGI protocol to the **WordPress** container on **port 9000**.
3. **WordPress** (running php-fpm) executes the PHP. To build the page it needs data — posts, users, settings — so it opens a SQL connection to the **MariaDB** container on **port 3306**.
4. **MariaDB** reads/writes its data, which physically lives on a **named volume** on the host so it survives container restarts. It returns rows to **WordPress**.
5. **WordPress** renders HTML and hands it back to **NGINX**, which encrypts it and sends it to the browser.

THE SINGLE MOST IMPORTANT DESIGN RULE

Only NGINX is reachable from outside. MariaDB and WordPress have **no published ports** — they are only reachable from inside the private Docker network. This is "defense in depth": even if someone reaches your host, they cannot talk to your database directly. Notice the diagram has exactly one red arrow crossing the host boundary.

1.5 What you will learn (the real syllabus)

Concept	Why the project forces you to learn it
Virtualization vs containerization	You run containers inside a VM and must explain in the README why they differ.
The Docker image/layer model	You write Dockerfiles and must keep them efficient and cache-friendly.
PID 1 & foreground processes	Forbidden hacks (<code>tail -f</code> , <code>sleep infinity</code>) force you to understand how a container's main process must behave.
Container networking & DNS	Services find each other by name on a user-defined bridge network.
Persistent storage	Named volumes keep your DB and site alive across rebuilds; you must contrast them with bind mounts.
TLS / HTTPS	NGINX must serve TLSv1.2/1.3 only — you generate a certificate and configure the protocol.
Secrets management	No passwords in Dockerfiles or Git; you use <code>.env</code> and Docker secrets.
Service orchestration	<code>docker-compose</code> wires everything together with dependencies and restart policies.
Build automation	A <code>Makefile</code> is the single entrypoint that builds and launches everything.

1.6 What "done" looks like (success criteria)

- ☐ A fresh `make` builds all images from your own Dockerfiles and starts the stack.
- ☐ Visiting `https://login.42.fr` shows a working WordPress site over HTTPS.
- ☐ The WordPress admin panel works; there are two DB users and the admin login does not contain "admin".
- ☐ Stopping and restarting the stack keeps all data (posts, uploads, DB) — persistence works.
- ☐ No container uses `latest` , runs an infinite-loop hack, or exposes ports it shouldn't.
- ☐ No password appears anywhere in the Git repository.
- ☐ The README, USER_DOC.md and DEV_DOC.md are present and explain the required comparisons.

CHAPTER 2

The mental models you need first

Before any Docker command makes sense, you need four ideas straight in your head: what a VM is, what a container is, why they differ, and the difference between an image and a container.

2.1 What a Virtual Machine actually is

A virtual machine emulates an **entire computer**. A piece of software called a **hypervisor** (VirtualBox, VMware, KVM) carves out fake CPU, fake RAM, fake disk and fake network from your real hardware, and on top of that fake hardware you install a **complete guest operating system** — its own Linux kernel, its own init system, its own everything. The guest OS believes it is running on a real machine and has no idea it is a guest.

This is powerful (total isolation, you can even run a different OS than the host) but heavy: every VM carries a full OS, takes minutes to boot, and eats gigabytes of RAM and disk before you've run a single useful program.

YOU ARE REQUIRED TO USE A VM FOR THIS PROJECT

Not because VMs and containers are the same, but because Docker needs a Linux kernel and the school wants a clean, disposable, reproducible Linux environment. So the layering is: **your laptop** → **a Linux VM** → **Docker** → **containers**. That is the "go deeper" joke made real.

2.2 What a container actually is

A container does **not** emulate hardware and does **not** ship its own kernel. A container is just **one or more normal Linux processes running on the host's kernel**, but tricked into believing they are alone on the machine. The trick is built from ordinary Linux kernel features you already half-know from minishell:

- **Namespaces** — the kernel can give a process its own private view of a resource. There is a PID namespace (the process sees itself as PID 1 and can't see host processes), a network namespace (its own interfaces and ports), a mount namespace (its own filesystem tree), a user namespace, and more. This is what produces *isolation*.
- **cgroups (control groups)** — the kernel can cap how much CPU, memory and I/O a group of processes may use. This is what produces *resource limits*.
- **Union/overlay filesystems** — the kernel can stack read-only layers and one writable layer into a single filesystem view. This is what makes images cheap and fast (more in chapter 3).

Put those together and you get a process that has its own filesystem, its own network, its own process table — but is still just a process scheduled by the same kernel as everything else. **That is a container**. No emulation, near-instant startup, megabytes not gigabytes.

2.3 VM vs Container, side by side

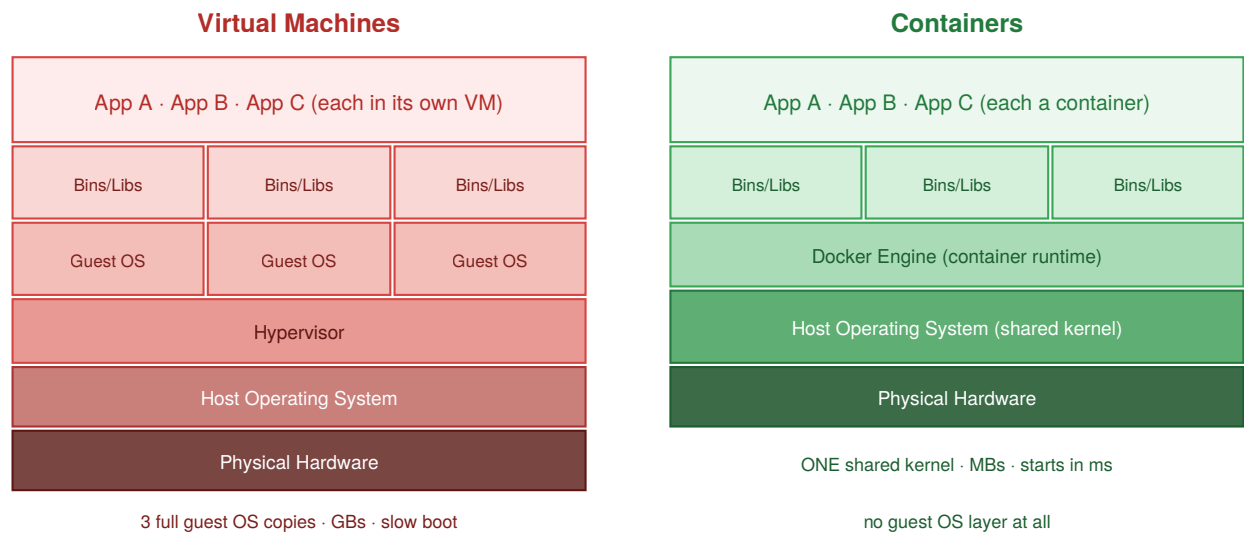


Figure 2.1 — The key visual difference: VMs each carry a full guest OS on top of a hypervisor; containers share the host kernel through the Docker engine. Fewer layers = lighter and faster.

Dimension	Virtual Machine	Container
Isolates by	Emulating hardware (hypervisor)	Kernel namespaces + cgroups
Ships its own kernel?	Yes, a full guest OS	No — shares the host kernel
Startup time	Seconds to minutes	Milliseconds
Size on disk	Gigabytes	Megabytes
Isolation strength	Very strong (separate kernels)	Strong, but shares one kernel
Best for	Running different OSES, hard security boundaries	Packaging & shipping apps reproducibly

README REQUIREMENT: "VIRTUAL MACHINES VS DOCKER"

Your README must contain exactly this comparison. The honest takeaway to write in your own words: a container is not a lighter VM — it's a fundamentally different mechanism. A VM virtualizes *hardware*; a container virtualizes the *operating system's* view for a process.

2.4 Images vs Containers — the class/instance idea

This trips up everyone at first, so anchor it to something you know from C++:

An **image** is like a **class**: a read-only blueprint — the filesystem and metadata that says "this is Alpine, with NGINX installed, configured this way, and when you start it run this command." It does nothing on its own; it just sits there.

A **container** is like an **object**: a running instance of an image. You can create many containers from one image, just as you instantiate many objects from one class. Each running container adds a thin writable layer on top of the read-only image.

```
# build an image (the blueprint) from a Dockerfile
docker build -t mariadb ./srcs/requirements/mariadb

# run a container (an instance) from that image
docker run mariadb
```

REMEMBER THE LIFECYCLE WORDS

You **build** an image. You **run** a container from it. You can **stop**, **start** and **remove** containers. Removing a container does not remove its image, and removing either does not (by default) remove a **volume** — that's the whole point of volumes, covered in chapter 5.

CHAPTER 3

Docker architecture, deep

When you type `docker run`, a surprising amount happens. Understanding the pieces makes error messages readable and makes you sound like you know what you're doing at the defense.

3.1 Docker is a client–server program

The `docker` command you type is just a **client**. It does almost nothing itself — it packages your request and sends it over a Unix socket to a long-running background **daemon** called `dockerd`. The daemon is what actually builds images, starts containers, manages networks and volumes. This separation is why Docker needs a running service and why you sometimes need `sudo` or membership in the `docker` group.

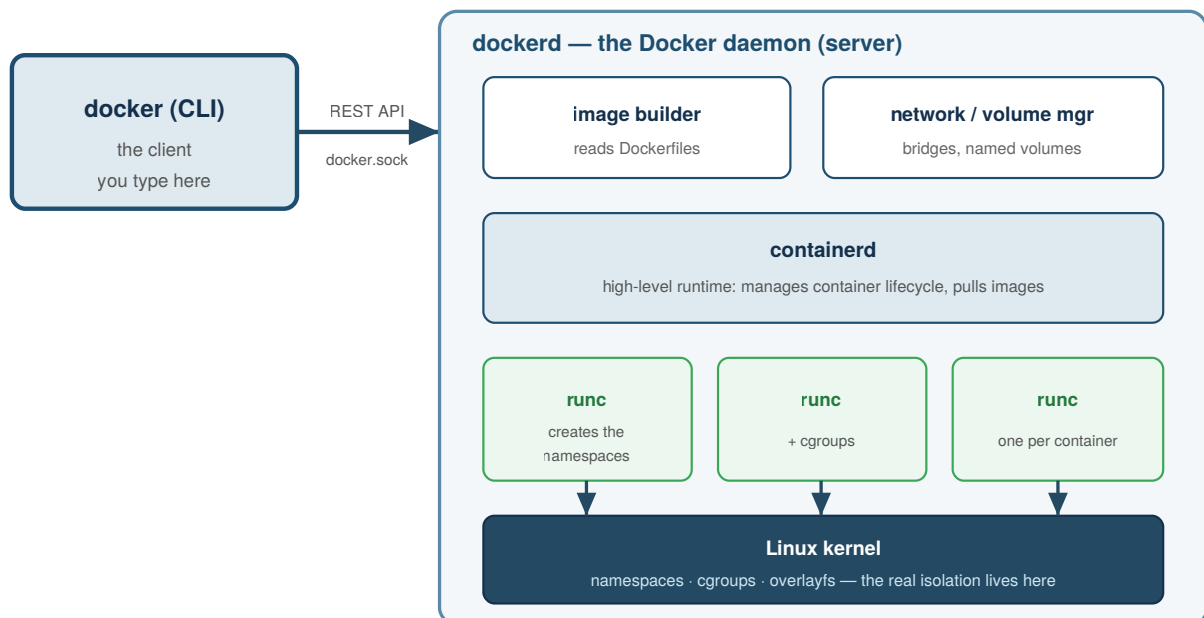


Figure 3.1 — The Docker stack. Your CLI talks to `dockerd`, which delegates running containers to `containerd`, which spawns one `runc` per container to set up the kernel namespaces and cgroups. The isolation itself is a kernel feature.

The layers explained

- **docker CLI** — the command you type; a thin client.
- **dockerd** — the daemon; builds images, exposes the API, manages networks/volumes.
- **containerd** — a lower-level runtime that manages the container lifecycle and image storage.
- **runc** — the lowest-level tool; it makes the actual syscalls to create namespaces and cgroups, then executes your container's process. One `runc` per container start.
- **the kernel** — where namespaces, cgroups and the overlay filesystem actually exist.

YOU WILL NOT CONFIGURE CONTAINERD OR RUNC DIRECTLY

You only ever talk to the CLI and write Dockerfiles/compose files. But knowing this chain exists means when a container "exits immediately," you know where to look — usually your container's main process (chapter 4), not Docker's plumbing.

3.2 The layered filesystem — why images are cheap

An image is not one big blob. It is a **stack of read-only layers**, and each instruction in your Dockerfile that changes the filesystem creates a new layer. The kernel's overlay filesystem merges them into one view.

Image = read-only layers

Container = image + 1 writable layer

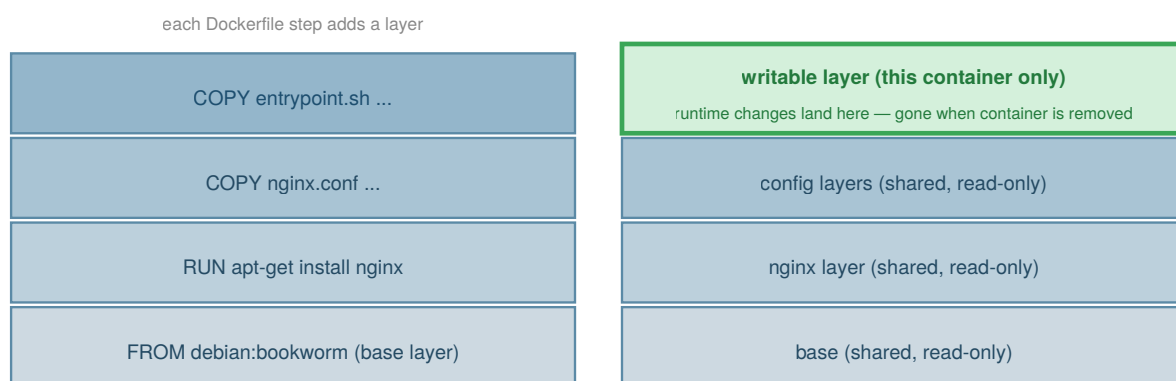


Figure 3.2 — Layers are shared and read-only; ten containers from one image share the same layers on disk. Each running container only adds its own thin writable layer. This is why you need **volumes** for anything that must survive.

THE CRUCIAL CONSEQUENCE

Anything written inside a container's writable layer **disappears when the container is removed**. Your database would be wiped on every `docker compose down`. That is the entire reason volumes exist (chapter 5).

3.3 The build cache — and why instruction order matters

Because each instruction is a layer, Docker **caches** them. On a rebuild, Docker reuses a cached layer as long as that instruction and everything before it are unchanged. The moment one instruction changes, that layer and **every layer after it** are rebuilt.

This is why experienced people put *rarely-changing* instructions early (install packages) and *frequently-changing* ones late (copy your config/scripts). Get the order wrong and every tiny edit re-runs a slow package install.

TWO RULES THE SUBJECT ENFORCES HERE

(1) The **latest** tag is forbidden — always pin a version like **debian:bookworm** or **alpine:3.19** so your build is reproducible. (2) Build from the **penultimate** (second-newest) stable Alpine or Debian. Check the current versions yourself and pick the one before the newest — and write the exact version, never **latest** .

CHAPTER 4

The Dockerfile, line by line

A Dockerfile is a recipe: a plain-text list of instructions that Docker executes top-to-bottom to build an image. You must write one per service. Here is every instruction you'll need.

4.1 The instructions you will use

Instruction	What it does	Notes for this project
FROM	Sets the base image — the starting filesystem.	Must be Alpine or Debian, a pinned penultimate version. Never latest .
RUN	Executes a command <i>at build time</i> , creating a new layer.	Used to install/configure packages. Each RUN is a layer.
COPY	Copies files from your build context into the image.	Used for config files and entrypoint scripts.
ARG	A build-time variable (only exists during build).	Can receive values from compose; not present at runtime.
ENV	Sets an environment variable available at runtime.	Use for non-secret config. Never hard-code passwords here.
EXPOSE	Documents which port the service listens on.	Documentation only — it does <i>not</i> publish the port.
WORKDIR	Sets the working directory for following instructions.	Cleaner than repeated cd .
USER	Switches the user for following instructions / runtime.	Good practice not to run everything as root where avoidable.
ENTRYPOINT	The fixed program the container runs.	Often a script that prepares state, then runs the real service.
CMD	Default arguments / the default command.	This is the foreground process that keeps the container alive.

4.2 The single most important concept: PID 1

Recall from minishell: every process has a PID, and PID 1 is special — it's the **init** process, the ancestor of everything. Inside a container's PID namespace, **your container's main process IS PID 1**. Whatever you put in **CMD** / **ENTRYPOINT** becomes PID 1.

A container lives exactly as long as its PID 1 lives. The instant PID 1 exits, the container stops. This one fact explains almost every rule in the subject:

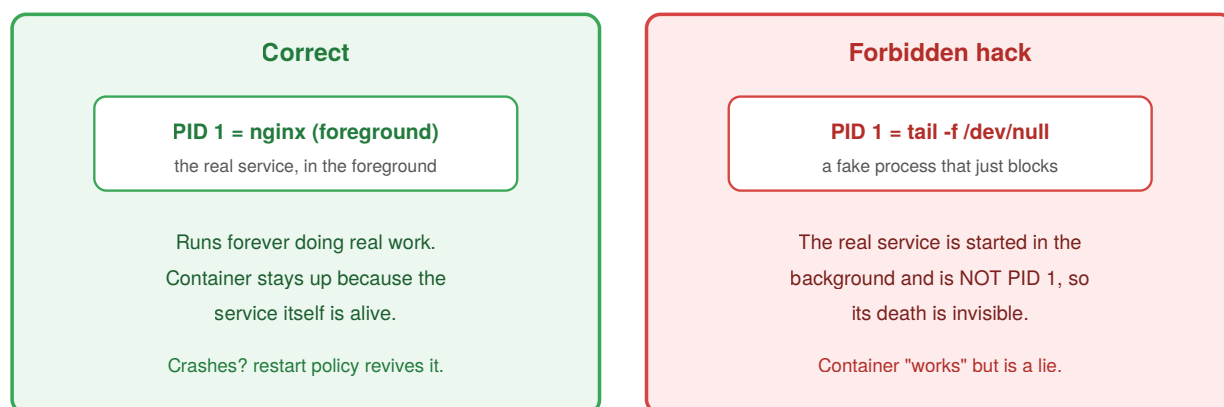


Figure 4.1 — Your service must BE PID 1, running in the foreground. Hacks like `tail -f`, `sleep infinity`, or `while true` keep the container alive with a fake PID 1 while the real service runs unsupervised. The subject forbids all of these.

4.3 Why those hacks are forbidden

Many tutorials cheat: they start the real service as a background daemon, then keep the container alive with a do-nothing command. The subject explicitly bans this because it defeats the purpose:

- If the real service crashes, the fake PID 1 keeps running, so Docker thinks everything is fine. Your `restart: always` policy never triggers because PID 1 never died. You have a "healthy" container serving nothing.
- Signals (like `docker stop` sending SIGTERM) go to the fake PID 1, not your service, so the service is never told to shut down cleanly.
- It shows you didn't understand how a container models a single service.

THE RULE, STATED PLAINLY

Each container's PID 1 must **be** the actual service, running in the **foreground**. NGINX, php-fpm and MariaDB all have a foreground mode — your job is to find the correct flag/command for each (chapter 7 points you at the concept, not the literal line).

4.4 Daemons, foreground, and "daemon off"

A **daemon** is a program that detaches from the terminal and runs in the background — exactly how services normally run on a normal Linux box. But inside a container that is backwards: if the service daemonizes, it forks a background process and the original (PID 1) exits, so the container dies immediately.

So for every service you must run it in **foreground mode**:

- **NGINX** has a directive that tells it not to daemonize (so it stays as PID 1).
- **php-fpm** has a foreground flag.
- **MariaDB** is launched via its server binary which runs in the foreground.

Your task is to read each program's documentation and find that foreground switch. This guide will not paste the exact flags — finding them and being able to explain them is the lesson.

4.5 ENTRYPOINT vs CMD, and entrypoint scripts

Often a service needs setup before it starts: create a database, generate a config from environment variables, create a user. The clean pattern is an **entrypoint script**: a small shell script that does the one-time setup and then hands control to the real service.

The critical detail is the last line of that script. If you just call the service normally, the script's shell stays PID 1 and the service becomes a child — wrong. Instead you use the shell builtin **exec**, which **replaces** the shell process with the service process (this is exactly **execve** from minishell — no fork, the new program takes over the same PID). Now the service is PID 1, in the foreground, correctly.

```
# conceptual shape of an entrypoint script - NOT the full answer
#!/bin/sh
# 1. one-time setup: init data dir if empty, create users from env vars
# 2. then, as the FINAL line, hand off to the real service:
exec <the-service-in-foreground> "$@"
```

CONNECT IT TO WHAT YOU KNOW

exec here is the same idea as **execve()** in minishell: it overlays the current process image with a new program, keeping the same PID. That is precisely why it makes your service become PID 1. You already understand this mechanism — you're just applying it in a new context.

4.6 A safe Dockerfile skeleton to reason about

This is the *shape* to internalize — the order and the kinds of instruction — not a copy-paste solution. You must fill in the real packages, configs and commands per service.

```
FROM debian:<pinned-penultimate-version>

# install only what THIS service needs (rarely changes - early for caching)
RUN apt-get update && apt-get install -y --no-install-recommends \
    <packages> \
    && rm -rf /var/lib/apt/lists/*

# bring in config and the entrypoint (changes often - later)
COPY conf/<service>.conf /etc/<...>/
COPY tools/entrypoint.sh /usr/local/bin/

EXPOSE <internal-port>          # documentation only

ENTRYPOINT ["entrypoint.sh"]    # setup, then exec the service
# or CMD [<"<service>", "<foreground-flag>"]
```

SELF-CHECK BEFORE YOU MOVE ON

Can you answer these without looking? (1) What becomes PID 1 in your container? (2) What happens the instant PID 1 exits? (3) Why does `exec` matter in an entrypoint script? (4) Why is `latest` banned? If yes, you've understood chapter 4.

CHAPTER 5

Networking & volumes

Two questions every multi-container app must answer: how do containers find and talk to each other, and how does data survive a container's death? Networks answer the first; volumes the second.

5.1 Container networking — the bridge network

When you create a **user-defined bridge network** and attach your containers to it, Docker does two beautiful things for you:

- It gives every container a private IP on that network so they can reach each other.
- It runs an internal **DNS** so containers can find each other *by service name* instead of by IP. So WordPress connects to a host literally called `mariadb`, and Docker resolves that name to the right container's IP — even if the IP changes on restart.

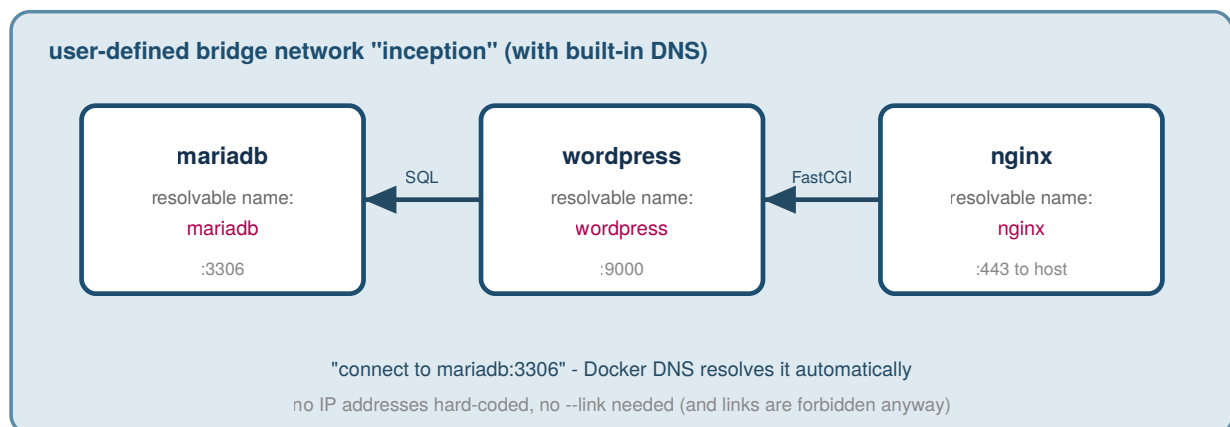


Figure 5.1 — On a user-defined bridge, containers reach each other by service name through Docker's built-in DNS. This is why the subject forbids `--link`, `links:` and `network: host` — the modern, correct way is a named network with name-based discovery.

Why the subject bans host networking and links

- **host networking** removes network isolation entirely — the container shares the host's network stack, so there's no separation and any container could grab any port. That destroys the whole isolation goal.
- **links** is the old, deprecated way to connect two containers. User-defined networks with DNS replaced it years ago. Using it signals outdated knowledge.

README REQUIREMENT: "DOCKER NETWORK VS HOST NETWORK"

In your own words: a user-defined bridge gives isolated, private, name-resolvable connectivity between your services while keeping the host protected; host networking flattens that boundary and is a security and design regression for this kind of stack.

5.2 Volumes — making data survive

Recall from chapter 3: a container's writable layer is destroyed when the container is removed. So your database would vanish on every rebuild. A **volume** is a piece of storage that lives *outside* the container's lifecycle, managed by Docker, and mounted into the container at a path. The container reads and writes there as normal, but the data persists independently.

Named volumes vs bind mounts

There are two ways to give a container persistent storage, and the subject is specific about which to use:

Named volume (required here)	Bind mount (not allowed for the two persistent stores)
Docker creates and manages the storage; you refer to it by a name.	You mount a specific host directory straight into the container.
Docker owns the location and lifecycle; portable and clean.	Tightly couples the container to one host path and its permissions.
Survives container removal; explicitly removed only when you ask.	Lives wherever you pointed it; behaves like a window onto the host FS.
Ideal for databases and app data managed by the service.	Ideal for live-editing source code during development.

THE PROJECT'S TWIST

The subject requires **named volumes**, but also requires their data to physically live in `/home/login/data` on the host. You achieve this by defining named volumes whose driver options point the storage at that host directory. The result is still a Docker-managed named volume (satisfying the rule) while letting you see the files at a known path. Read the compose **volumes:** driver-options documentation to discover how — that investigation is part of the exercise.

README REQUIREMENT: "DOCKER VOLUMES VS BIND MOUNTS"

Write the trade-off in your own words: named volumes are Docker-managed, portable and the right choice for service-owned persistent data; bind mounts expose a host path directly and are better for development where you want changes on disk to appear live inside the container.

Two volumes, mapped to the architecture

- **DB volume** → mounted into MariaDB at its data directory, so all your tables persist.

- **WordPress files volume** → mounted into the WordPress container (and read by NGINX), holding the PHP files, themes and uploads.

CHAPTER 6

docker-compose

Running three containers, a network and two volumes with individual `docker run` commands would be painful and error-prone. `docker-compose` lets you declare the entire stack in one YAML file and bring it all up with a single command.

6.1 What compose gives you

- One file (`docker-compose.yml`) describes every service, network and volume.
- `docker compose up --build` builds all images and starts everything in the right order.
- It creates the network and volumes for you and attaches each service.
- It applies restart policies, environment variables, secrets and dependencies declaratively.

"DOCKER COMPOSE" VS "DOCKER-COMPOSE"

Older systems use the standalone `docker-compose` (with a hyphen). Modern Docker ships it as a plugin invoked as `docker compose` (a space). They're functionally equivalent for this project; use whichever your VM provides and stay consistent in your Makefile.

6.2 Anatomy of the compose file

A compose file has a few top-level keys. Here is the skeleton to understand — fill in the real details yourself:

```

services:
  mariadb:
    build: ./requirements/mariadb      # path to that service's Dockerfile
    image: mariadb                     # image name == service name (required)
    container_name: mariadb
    restart: always                    # revive on crash (required)
    networks: [ inception ]
    volumes: [ db_data:/var/lib/mysql ]
    env_file: .env                     # inject non-secret config
    secrets: [ ... ]                  # inject passwords safely
    # NOTE: no "ports:" - the DB must NOT be reachable from the host

  wordpress:
    build: ./requirements/wordpress
    image: wordpress
    depends_on: [ mariadb ]            # start order hint

  nginx:
    build: ./requirements/nginx
    image: nginx
    ports: [ "443:443" ]               # the ONLY published port in the stack
    depends_on: [ wordpress ]

networks:
  inception:
    driver: bridge

volumes:
  db_data:    # named volume - driver options point at /home/login/data/...
  wp_data:

secrets:
  # declare files holding passwords, read from ./secrets/

```

THINGS THE SUBJECT WILL CHECK IN THIS FILE

The `networks:` section must be present (not implicit). No `links:`, no host networking. Only NGINX has a `ports:` entry, and only 443. Every image name equals its service name. Every service has `restart`. No `latest` tags anywhere.

6.3 Environment variables and the `.env` file

Hard-coding values (domain name, database name, usernames) inside Dockerfiles is rigid and, for secrets, dangerous. Instead, configuration goes into environment variables, and the non-secret ones live in a `.env` file that compose reads automatically.

```
# srcs/.env - non-secret configuration
DOMAIN_NAME=login.42.fr
MYSQL_DATABASE=wordpress
MYSQL_USER=wp_user
WP_ADMIN_USER=<a-name-without-"admin"-in-it>
# Passwords do NOT belong here in the cleanest setup - use secrets.
```

README REQUIREMENT: "SECRETS VS ENVIRONMENT VARIABLES"

Environment variables are convenient for non-sensitive config, but they're easy to leak: they show up in `docker inspect`, in process listings, and in logs. **Secrets** are designed for sensitive values — they're mounted as files inside the container (typically under `/run/secrets/`), not exposed in the environment, and the files holding them stay out of Git.

6.4 Docker secrets — keeping passwords out of Git

The subject is severe about this: **any password found in your Git repository fails the project**. The mechanism to avoid that:

1. Put each sensitive value in its own file under a `secrets/` directory (e.g. `db_password.txt`, `db_root_password.txt`, `credentials.txt`).
2. Declare those files as `secrets` in the compose file and grant them to the services that need them.
3. Inside the container, read the secret from its file (often via a `..._FILE` environment variable that points at `/run/secrets/...`), rather than passing the password directly.
4. Add `secrets/` and `.env` to `.gitignore` so they are never committed.

DO THIS ON DAY ONE

Create your `.gitignore` *before* you ever create a file with a password in it. The single most common way people fail this project is committing a secret early, because even if you delete it later, it lives forever in the Git history. If that happens, you must rewrite history or start a fresh repository.



Figure 6.1 — Secrets flow from gitignored files on the host into the container as mounted files under `/run/secrets/`, never touching the environment, the image, or Git history.

CHAPTER 7

The three mandatory services

Now we connect every concept to the actual containers you build. For each service: what it is, what it must do, the foreground concern, and the questions to research — never the literal answer file.

7.1 How a single page load travels the whole stack

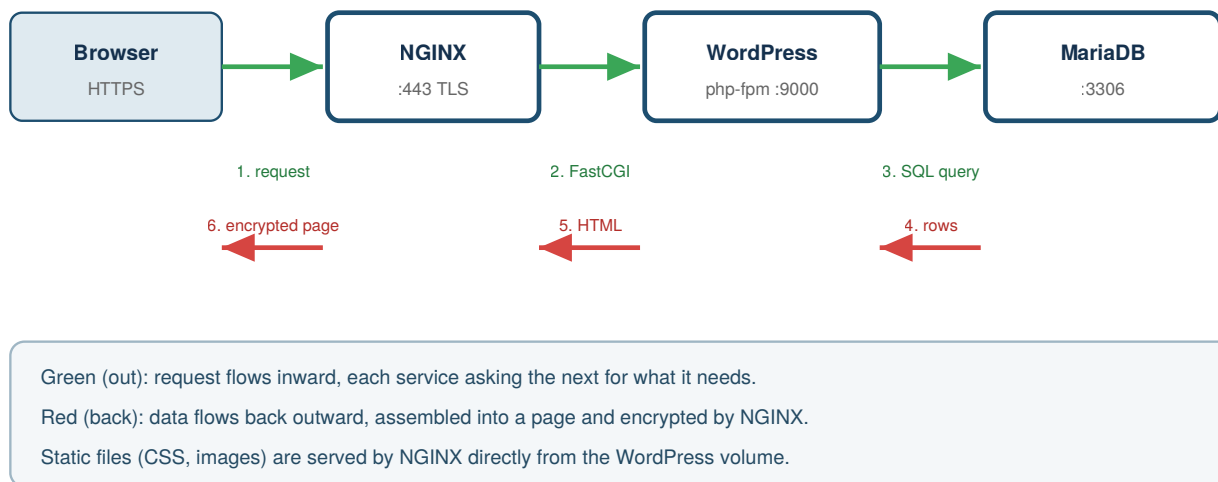


Figure 7.1 — One dynamic page load. Build order is the reverse: MariaDB must be ready before WordPress can initialize, and WordPress before NGINX is useful.

7.2 MariaDB — the database

What it is

MariaDB is a relational SQL database (a community fork of MySQL). It stores everything WordPress remembers: posts, pages, users, settings, comments. It is the foundation — build it first.

What your container must do

- Install MariaDB server on your pinned Alpine/Debian base.
- On first launch, initialize the data directory, create the WordPress database, and create the **two required users** — one being the admin (whose name must not contain admin/administrator).
- Read passwords from secrets, not from hard-coded strings.
- Persist its data directory onto the DB named volume.
- Run the server in the **foreground** as PID 1 — no `tail -f`.
- **Not** publish port 3306 to the host. Only WordPress (on the network) talks to it.

Research these, don't copy them

- How do you run an initialization SQL step only the first time (when the volume is empty)?
- Which command runs the MariaDB server in the foreground for PID 1?
- How do you bind the server to listen on the container's network interface so WordPress can reach it?
- How do you make the root and user passwords come from secret files?

IDEMPOTENCY

Your entrypoint should detect whether the database already exists (because the volume persisted from a previous run) and skip re-initialization. Otherwise a restart would try to recreate users and fail. "Run setup only if not already set up" is the pattern to implement.

7.3 WordPress + php-fpm — the application

What it is

WordPress is a PHP web application. PHP code doesn't run by itself — it needs an interpreter. **php-fpm** (FastCGI Process Manager) is a long-running PHP runtime that listens on a port (9000) and executes PHP scripts on request. NGINX hands PHP requests to php-fpm and gets back the generated HTML.

What your container must do

- Install PHP, php-fpm, and the PHP extensions WordPress needs (including the MySQL/MariaDB driver).
- Download the WordPress source and place it on the WordPress named volume.
- Configure WordPress to connect to the `mysqli` host using the DB name/user/password from env+secrets.
- Programmatically install WordPress and create the admin and second user (a tool exists for scripting WordPress setup — find it).
- Configure php-fpm to listen on the network (port 9000) so NGINX can reach it.
- Run php-fpm in the **foreground** as PID 1.
- Contain **no NGINX** — this container is PHP only.

Research these

- What command-line tool lets you install and configure WordPress without clicking through the web installer?
- Which php-fpm config directive makes it listen on a TCP socket reachable from another container?
- What's the foreground flag for php-fpm?
- How do you make WordPress wait until MariaDB is actually accepting connections (not just "started")?

THE ADMIN USERNAME TRAP

The subject forbids admin names containing "admin"/"administrator" in any casing or with suffixes (`admin` , `Administrator` , `admin-123` ...). Pick something neutral and set it via your env file. Evaluators check this specifically.

7.4 NGINX + TLS — the gateway

What it is

NGINX is a high-performance web server and reverse proxy. Here it plays two roles: it terminates TLS (does the HTTPS encryption) and it reverse-proxies PHP requests to php-fpm. It is the only container exposed to the outside world.

What your container must do

- Install NGINX on your pinned base.
- Generate a TLS certificate (self-signed is fine for this project) and configure NGINX to use it.
- Accept connections on **port 443 only**, allowing **TLSv1.2 or TLSv1.3 only** (explicitly disable older protocols).
- Serve static files from the WordPress volume directly; forward `.php` requests to `wordpress:9000` over FastCGI.
- Run NGINX in the foreground as PID 1 (there's a directive to disable daemonizing).

Research these

- How do you generate a self-signed certificate with the right Common Name for `login.42.fr` ?
- Which NGINX directives set `ssl_protocols` to restrict to TLSv1.2/1.3?
- What is the correct `fastcgi_pass` target and the FastCGI parameters for a PHP app?
- Which directive keeps NGINX in the foreground?

WHY TLS IS MANDATORY HERE

HTTP sends everything in plaintext — anyone on the network can read passwords and content. TLS encrypts the channel. The subject pins TLSv1.2/1.3 because older versions (SSLv3, TLSv1.0/1.1) have known vulnerabilities. Restricting protocols is a real-world security best practice you're learning to apply.

7.5 The domain name

You must make `login.42.fr` resolve to your VM's local IP. This is done by adding an entry to the host's `/etc/hosts` file mapping your domain to `127.0.0.1` (or the VM's IP). That's a single line; understand that DNS resolution checks `/etc/hosts` before going to real DNS servers, which is why this works locally.

CHAPTER 8

The Makefile & the host VM

8.1 The Makefile is your single entrypoint

You already know `make` from your C projects: it reads a `Makefile` of targets and recipes. Here it has nothing to compile — it simply orchestrates Docker. The subject requires a Makefile at the project root that sets up the entire application via `docker-compose.yml`.

Typical targets to provide (you decide the exact recipes):

Target	What it should do
<code>all / up</code>	Create the host data directories if needed, then build and start the stack.
<code>build</code>	Build the images without starting (optional).
<code>down</code>	Stop and remove the containers (keep volumes).
<code>clean</code>	Stop everything and remove containers/images.
<code>fclean</code>	Full reset — also remove volumes and the host data directory.
<code>re</code>	<code>fclean</code> then <code>all</code> — rebuild from scratch.

A SUBTLE BUT IMPORTANT DETAIL

Your named volumes point at `/home/login/data/...`. Docker won't create that host directory for you when using the local driver with a bind-style device option — so your Makefile (or an entrypoint) often needs to `mkdir -p` those directories before `compose up`. Discovering this the hard way (volume fails to mount) is part of the learning; now you know where to look.

DON'T HIDE REAL RECIPES FROM YOURSELF

Resist copying a Makefile wholesale. At the defense you may be asked "what does `fclean` do and why does `re` depend on it?" Write each recipe so you can explain it.

8.2 Setting up the VM

The whole project runs in a Linux VM. The high-level path:

1. Install a hypervisor (VirtualBox is common at 42) and create a VM with a Linux distro.
2. Give it enough disk (Docker images add up) and a user account matching your login.
3. Install Docker Engine and the compose plugin inside the VM, and add your user to the `docker` group.
4. Clone your repo inside the VM and work entirely there — the evaluation happens in this environment.

WHY INSIDE A VM AND NOT YOUR HOST DIRECTLY?

Isolation and reproducibility: a disposable VM means a broken experiment never touches your real machine, and everyone evaluates in an equivalent clean environment. It also guarantees a Linux kernel for Docker if your host is macOS or Windows.

CHAPTER 9

The bonus stack

*The bonus is only graded if the mandatory part is **perfect**. Each bonus service follows the same rules: its own Dockerfile, its own container, its own volume if needed, attached to the same network.*

READ THIS BEFORE STARTING BONUS

"Perfect" means the mandatory part fully works with zero malfunctions. If a single mandatory requirement fails, the entire bonus is ignored. Finish and harden the mandatory stack completely before adding anything here.

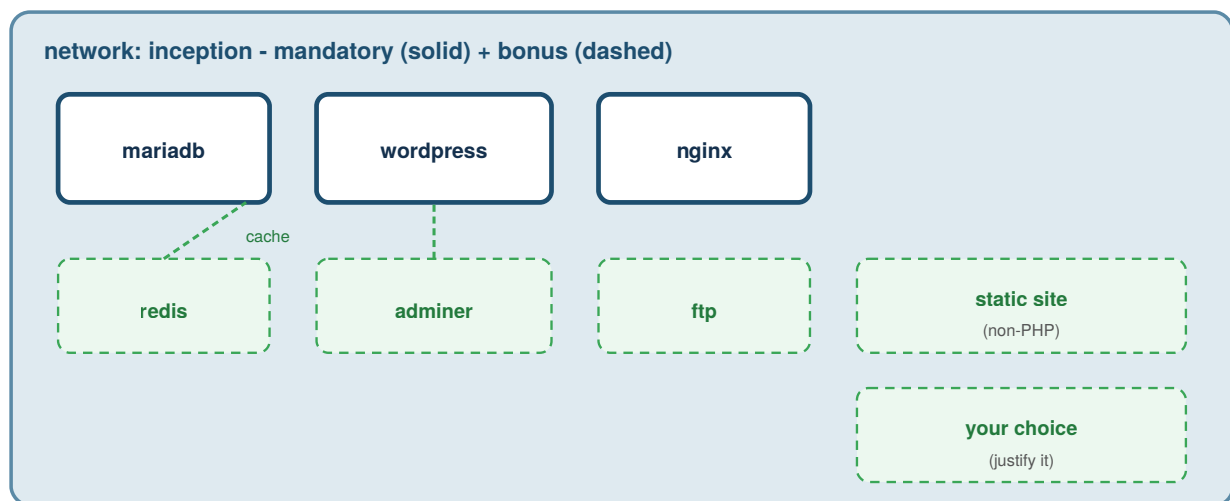


Figure 9.1 — Bonus services join the same network and follow the same one-service-per-container rule.

9.1 Redis cache

What: Redis is an in-memory key-value store. WordPress can use it as an object cache so repeated database queries are served from RAM instead of hitting MariaDB every time — faster pages, less DB load.

Concepts to learn: what caching is and why it helps; how WordPress's object-cache plugin talks to a Redis backend; running `redis-server` in the foreground with a config that enables it on the network. **Research:** which WordPress plugin enables Redis object caching, and how to point it at the `redis` host.

9.2 FTP server

What: An FTP server that exposes the WordPress files volume, so you can upload/download site files over FTP.

Concepts: FTP's control vs data connections and passive mode (which matters a lot inside containers because of port ranges); mounting the same WordPress volume into this container. **Research:** a lightweight FTP daemon (e.g. vsftpd).

or pure-ftpd), how to create an FTP user from secrets, and how to configure passive-port ranges and open them in compose.

9.3 A static website (non-PHP)

What: A small site in any language except PHP — a portfolio or showcase. Good simple choices: plain HTML/CSS/JS served by a tiny web server, or something in Python/Go/Node. **Concepts:** serving static content; choosing a minimal base image; running the server in the foreground. **Research:** the smallest correct way to serve your chosen stack and how to route to it (its own port, since the subject allows extra ports for bonus).

9.4 Adminer

What: Adminer is a single-file PHP database admin tool — a web UI to inspect and edit your MariaDB. **Concepts:** it's another PHP app, so it also needs php-fpm or a built-in server; it connects to `mariadb` on the network. **Research:** how to fetch the Adminer file and serve it, and how to expose it safely (it can see your whole database).

9.5 A service of your choice

Pick something genuinely useful and be ready to justify it at the defense. Common picks: a monitoring dashboard (e.g. a metrics/visualization tool), a small mail catcher, or a personal app. The grading weight is on your *justification* and that it follows all the same rules (own Dockerfile, foreground PID 1, no `latest`, secrets handled properly).

BONUS GOLDEN RULE

Every bonus container obeys the exact same constraints as the mandatory ones. The fastest way to lose bonus points is to slip a `latest` tag, a hard-coded password, or a `tail -f` into a bonus service because "it's just bonus." It isn't — same rules everywhere.

CHAPTER 10

The command reference

Every command you'll realistically touch, grouped by what you're trying to do. Run each one on something real while you read — that's how they stick.

10.1 Images & building

Command	What it does
<code>docker build -t name path</code>	Build an image named <code>name</code> from the Dockerfile at <code>path</code> .
<code>docker images</code>	List images on the machine (look for stray <code>latest</code> tags).
<code>docker rmi name</code>	Remove an image.
<code>docker history name</code>	Show an image's layers — great for understanding your Dockerfile's cost.

10.2 Containers & lifecycle

Command	What it does
<code>docker ps</code>	List running containers. Add <code>-a</code> to include stopped ones.
<code>docker run image</code>	Create and start a container from an image.
<code>docker stop / start name</code>	Stop (SIGTERM then SIGKILL) or start a container.
<code>docker rm name</code>	Remove a stopped container.
<code>docker logs name</code>	Show a container's stdout/stderr — your #1 debugging tool.
<code>docker logs -f name</code>	Follow logs live.
<code>docker exec -it name sh</code>	Open a shell <i>inside</i> a running container to inspect it.
<code>docker inspect name</code>	Dump full JSON config — IPs, mounts, env, restart policy.

10.3 Compose (the commands you'll use most)

Command	What it does
<code>docker compose up</code>	Build (if needed) and start the whole stack.
<code>docker compose up -d --build</code>	Force rebuild and run detached (in the background).
<code>docker compose down</code>	Stop and remove containers + network (volumes survive).
<code>docker compose down -v</code>	Also remove volumes — wipes your data. Use deliberately.
<code>docker compose ps</code>	Status of the stack's services.
<code>docker compose logs -f service</code>	Follow one service's logs.
<code>docker compose build service</code>	Rebuild a single service's image.

10.4 Networks & volumes

Command	What it does
<code>docker network ls</code>	List networks.
<code>docker network inspect name</code>	See which containers are attached and their IPs.
<code>docker volume ls</code>	List volumes.
<code>docker volume inspect name</code>	See where a volume's data actually lives on the host.

10.5 Cleanup (use carefully)

Command	What it does
<code>docker system prune</code>	Remove stopped containers, unused networks, dangling images.
<code>docker system prune -a --volumes</code>	Aggressive full cleanup including volumes — destroys data.

DEBUGGING WORKFLOW TO MEMORIZE

Container won't stay up? Run this loop: `docker compose up` (watch it in the foreground) → read the error → `docker compose logs service` for the failing one → `docker exec -it service sh` to poke around inside if it's still alive → fix the Dockerfile/config → rebuild. 90% of bugs are a wrong path, a missing foreground flag, or a service that exited because PID 1 died.

CHAPTER 11

A suggested roadmap & the defense

Build in dependency order, one service at a time, verifying each before adding the next. Trying to write all three Dockerfiles and the compose file at once is the path to confusion.

11.1 The order to actually build

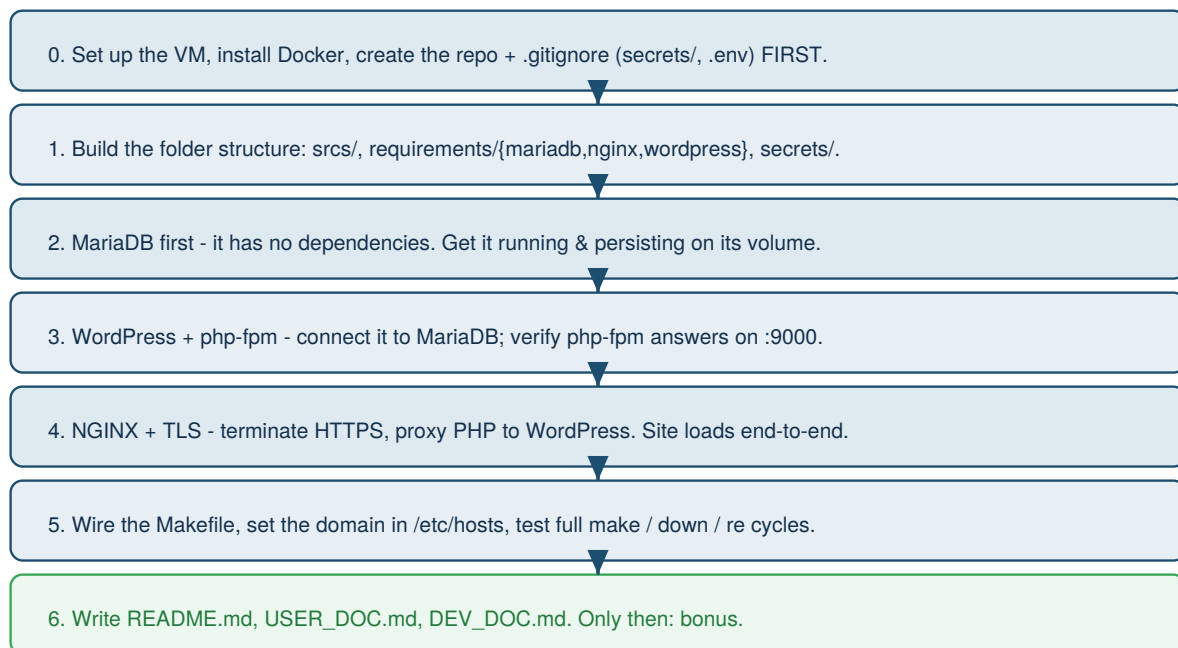


Figure 11.1 — Build in dependency order. Each step is verifiable on its own before you move on, which makes debugging tractable.

11.2 Verify each step before moving on

- ☐ MariaDB: `docker compose up mariadb` → check logs say it's ready → the DB persists after down/up.
- ☐ WordPress: connects to the DB without errors; php-fpm process is PID 1 and listening on 9000.
- ☐ NGINX: `https://login.42.fr` loads; `http://` and old TLS versions are refused.
- ☐ Persistence: create a post, run `make down` then `make` — the post is still there.
- ☐ Security sweep: `git log -p` shows no passwords; no `latest`; only 443 published.

11.3 Required documentation

The subject requires three Markdown files at the repo root. Treat them as a chance to consolidate your understanding, and write them in your own words:

- **README.md** — first line italic credit line; Description, Instructions, Resources (including *how you used AI*), and a Project description with the four required comparisons (VM vs Docker, Secrets vs Env Vars, Docker Network vs Host, Volumes vs Bind Mounts). English only.
- **USER_DOC.md** — for an end user/admin: what the stack provides, how to start/stop it, how to reach the site and admin panel, where credentials live, how to check services are healthy.
- **DEV_DOC.md** — for a developer: setting up from scratch (prereqs, configs, secrets), building/launching via Makefile + compose, useful container/volume commands, where data persists.

11.4 Preparing for the defense

The evaluation may ask you to make a small live change and to explain any part of your project. Be ready to answer, out loud, questions like:

- Why is a container not a VM? What does each isolate?
- What is PID 1 in your NGINX container, and what happens if it exits?
- Why is `tail -f` forbidden, and what did you do instead?
- How do your containers find each other without IP addresses?
- Why named volumes and not bind mounts here? Where does your data physically live?
- How do you keep passwords out of Git? Show me.
- Why TLSv1.2/1.3 only?
- Walk me through one page load from browser to database and back.

THE HONEST FINISH LINE

If your friend can answer every question in 11.4 from understanding — not memorization — then this guide did its job and the project will hold up under any defense. The code is the easy part once the concepts are solid. Build it slowly, break it on purpose, read the logs, and explain it to a peer before the evaluation. That is exactly the workflow the subject's AI-instructions chapter is asking for.

END OF GUIDE

Now go build it — one container at a time, and one layer deeper than you think you need to.